

School of Science, Computing and Engineering Technologies
Swinburne University of Technology
SWE30003 Software Architectures and Design
Tutorial Weeks 7-8

(OO Design Exercise II)

Please carefully read the (incomplete) specification of an automated teller machine (ATM) given on the following page.

For the purpose of this tutorial, you can assume that all hardware devices mentioned in the specification have a suitable API that can be used from the software to be developed. For questions 1 to 6 you can also assume that the ATM software will be “stand-alone”, that is, it does not need to communicate with any external servers (or similar).

Form groups to three to four students and discuss the following questions (note the difference in the following process concerning the identification and allocation of responsibilities, compared to that in the last tutorial exercise):

1. Identify the objects and classes that are needed in an object-oriented solution.

=> Candidate classes (Hardware Interface):

- CardReader: Reads card data; retains or ejects the card.
- Display: Shows menus, instructions, and balances to the user.
- Keypad: Captures user inputs (PINs, amounts, selections).
- CashDispenser: Counts and dispenses physical cash notes.
- ReceiptPrinter: Prints transaction details onto paper.

=> System Logic (Controller):

- ATMController: Orchestrates the main lifecycle and system states.
- SessionManager: Tracks a single user session from card insertion to timeout.

=> Core Data (Entities)

- BankDatabase: Acts as the local repository for all data in this stand-alone setup.
- Account: Stores account details and handles balance updates.
- Card: Holds linked account numbers and PIN verification data.
- Transaction: Base class handling timestamps and IDs. (*Subclasses: WithdrawalTransaction, BalanceInquiryTransaction*)

2. Identify all responsibilities that the ATM's software system needs to perform.

=> User Interaction & I/O Management

- Capture Input: Read card data upon insertion and capture numeric inputs (PIN, amounts, menu choices) via the keypad.
- Provide Feedback: Display menus, operational prompts, balances, and error messages on the screen.
- Produce Outputs: Control the physical dispensing of cash notes and print paper transaction receipts.

=> Session & Security Control

- Session Lifecycle: Detect card insertion to initiate a session and manage timeouts or manual cancellations to terminate it.
- Authentication: Verify the entered PIN against the local account/card data to authorize access.
- Card Management: Securely retain the card during the transaction and eject it safely upon session completion or invalid attempts.

=> Transaction & Data Processing

- Business Logic Execution: Process specific transaction requests, such as withdrawals or balance inquiries.
- Validation Rules: Check if the requested withdrawal amount is available within the user's account balance and if the physical cash dispenser holds enough notes.
- Data Persistence: Directly update and maintain local account balances and log transaction history records within the stand-alone system.

3. Assign each responsibility to one of the identified classes. Please attempt to distribute the responsibilities as evenly as possible throughout the system.

=> ATMController

- Collaborates with CardReader to detect card insertion/ejection.
- Collaborates with SessionManager to validate user state flows.

=> SessionManager

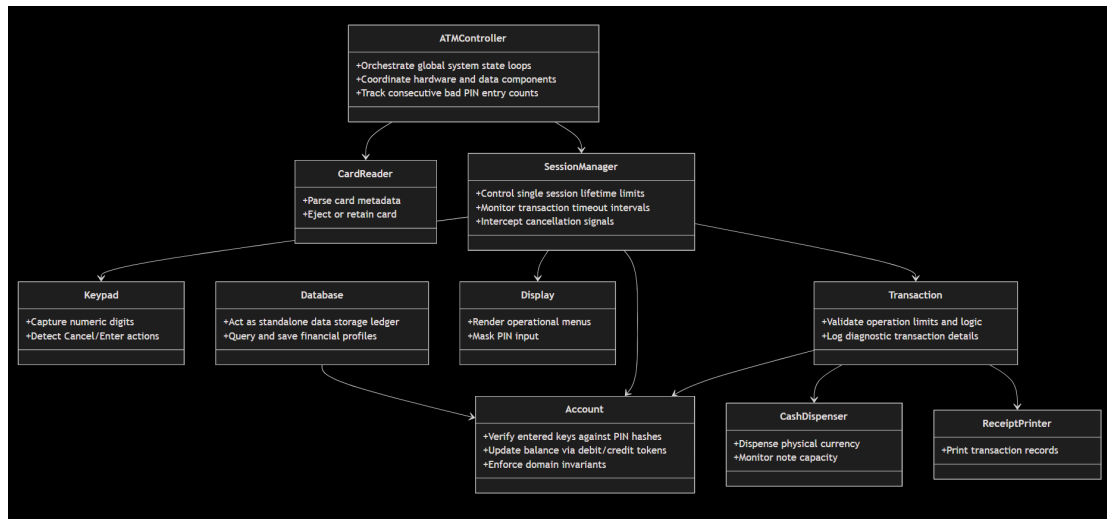
- Collaborates with Keypad and Display to handle PIN entry and main menu navigation.
- Collaborates with Account to authenticate the user's PIN.
- Collaborates with Transaction to execute a chosen service.

=> Transaction (and subclasses: Withdrawal, Deposit, Transfer, Inquiry)

- Collaborates with Account to check limits and update balances.
- Collaborates with CashDispenser to verify and distribute paper notes.
- Collaborates with ReceiptPrinter to print transaction activity records.

=> BankDatabase

- Collaborates with Account to pull data and persist updated financial balances locally.



4. For each class, identify possible collaborations that are needed to perform its responsibilities.

=> ATMController

- when a physical card is inserted or ejected.
- *Collaborates with:* SessionManager to initialize a fresh user state flow, hand off session control, or terminate operations upon failure.

SessionManager

- *Collaborates with:* Keypad and Display to drive UI navigation menus, capture user numeric entries, and mask PIN digits.
- *Collaborates with:* Account to forward input tokens and verify if the entered PIN matches the security key.
- *Collaborates with:* Transaction to instantiate and trigger the execution of a chosen banking service.

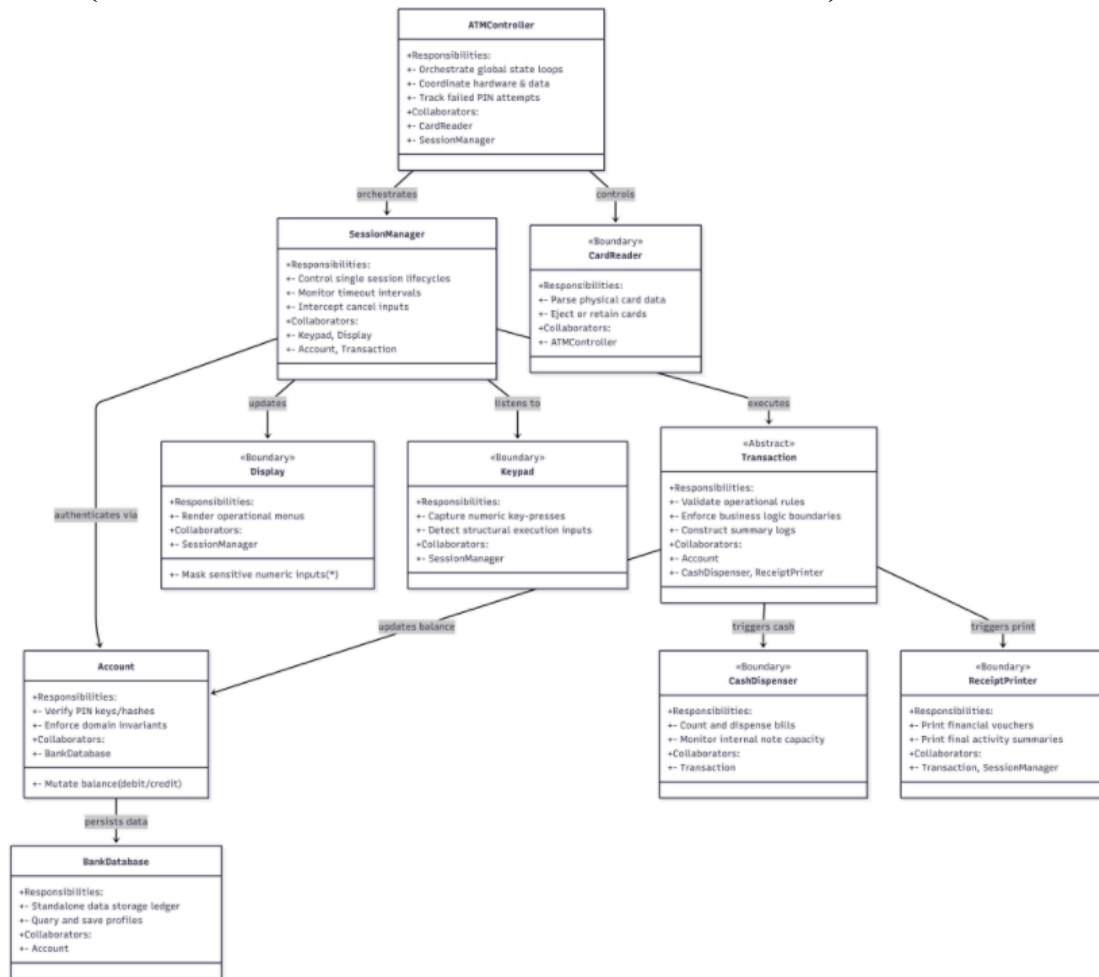
Transaction (and subclasses like WithdrawalTransaction)

- *Collaborates with:* Account to check core domain constraints (e.g., sufficient balance checking) and apply balance mutations directly.
- *Collaborates with:* CashDispenser to check available internal notes and command the hardware to physically distribute paper currency.
- *Collaborates with:* ReceiptPrinter to forward transaction logs and data blocks for physical paper printing.

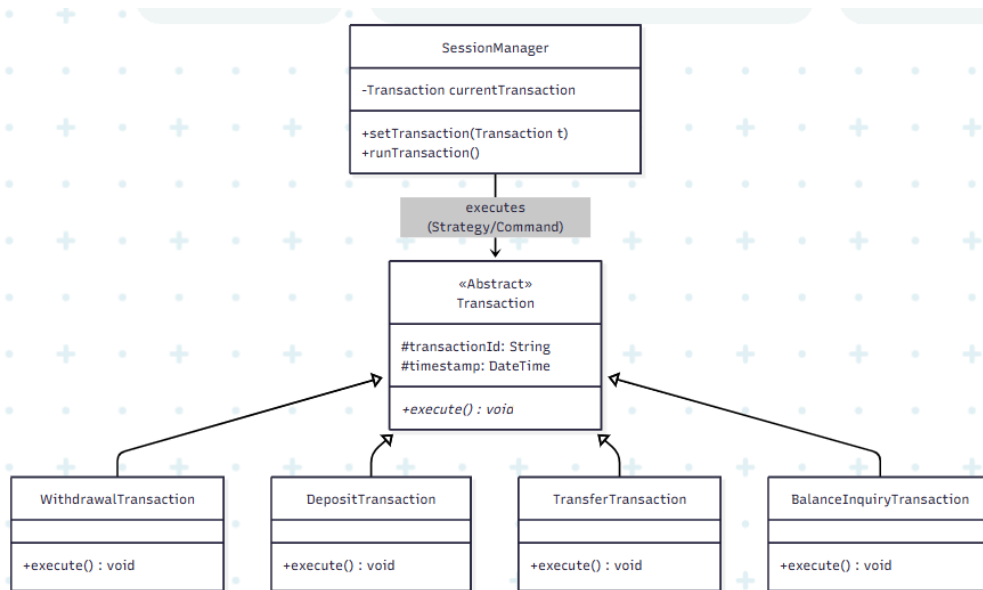
BankDatabase

- *Collaborates with:* **Account** to query and fetch saved entity payloads based on card info, and persist updated financial balances locally
- *Collaborates with:* **CardReader** to detect hardware events .

5. Document all identified classes, including their responsibilities and collaborations, using Class-Responsibility-Collaboration (CRC) cards. Please also include a brief description of the purpose of each class (that could be written on the back of each card).



6. Where applicable, identify further relationships between classes and organize the corresponding classes into class hierarchies. Also, highlight the application of any known design patterns in your design.



7. How would you go about ensuring that any transactions that change the balance of accounts are reflected across all of the bank's ATMs? Discuss the implications of your solution approach and provide a rationale of why the approach was chosen.

=> For synchronizing balance changes across all ATMs, the local database is replaced by a secure network client connecting to a centralized Core Banking Server. When a transaction occurs, the ATM sends a remote call to this central server, which applies pessimistic locking to isolate the account record and instantly eliminate dual-withdrawal race conditions. This approach prioritizes Consistency and Partition Tolerance (CP) from the CAP Theorem because financial systems require absolute real-time ACID compliance; thus, if the network link fails, the ATM cannot fallback on local updates and must immediately go "Out of Service" to prevent accidental overdrafts.

.....
8. Any questions from weekly submissions? => Currently is not

An automated teller machine (ATM) is a machine through which bank customers can perform a number of common financial transactions. An ATM machine generally consists of a display screen, a bankcard reader, numeric and special input keys, a money dispenser slot, a deposit slot and a receipt printer.

When the ATM machine is idle, a greeting message is displayed. The keys as well as the deposit slot will remain inactive until a bankcard has been entered into the bankcard reader.

When a bankcard is inserted into the bankcard reader, the reader attempts to read this card. If the card cannot be read, the user is informed that the card is unreadable and the card is ejected.

If the card is readable, the user is asked to enter a personal identification number (PIN). The user is given feedback as to the number of digits entered at the numeric keypad, but not the specific digits entered. If the PIN is entered correctly, the user is shown the main menu (described below). If the PIN is entered incorrectly, the user is

given up to two additional chances to enter the PIN correctly. Failure to do so on the third attempt causes the ATM machine to retain the bankcard and giving the user instructions how to retrieve the bankcard. This is only possible by going to dedicated branches of the corresponding financial institution.

The main menu of the ATM machine contains a list of transactions that the user can perform. These transactions are:

*Deposit funds into an account,
Withdraw funds from an account,
Transfer fund from one account into another account, and
Query the balance of an account.*

The user can select any of these transactions and specify all the relevant information. Once a transaction has been completed, the system returns to the main menu.

At any time after reaching the main menu and before completing any of the transactions, the user may press the cancel key. The transaction being specified (if there is one) is terminated, the user's card is returned, a summary of all transactions (if there were any) is printed, and the machine again becomes idle.